

COS214 Prac3

JA Rafudeen – u25011503

CL Botha – u25017749

SL Dlamini – u25141300

1.1) Our event is a Film Festival, called the South African National Film Festival. It is a place for aspiring filmmakers in South Africa to show off their work, for directors to host panels where they talk about their films, and for nominated films to be given awards. It can take place inside a cinema, a theatre, an outdoor venue, or an area that holds any of those, such as a mall (groupings that are Composite). Within those venues there can be a screening area, a ticket desk, an information desk, a place to buy food, or a place to host the special events such as the awards show (concrete event units).

1.2)

Composite participants:

Component – EventComponent

Composite – FilmShowing, SpecialEvent, Zone, OutdoorVenue, Cinema, Theatre

Leaf – ScreeningArea, TicketDesk, InformationDesk, FoodArea, SpecialEventArea

Observer participants:

Observer – Event

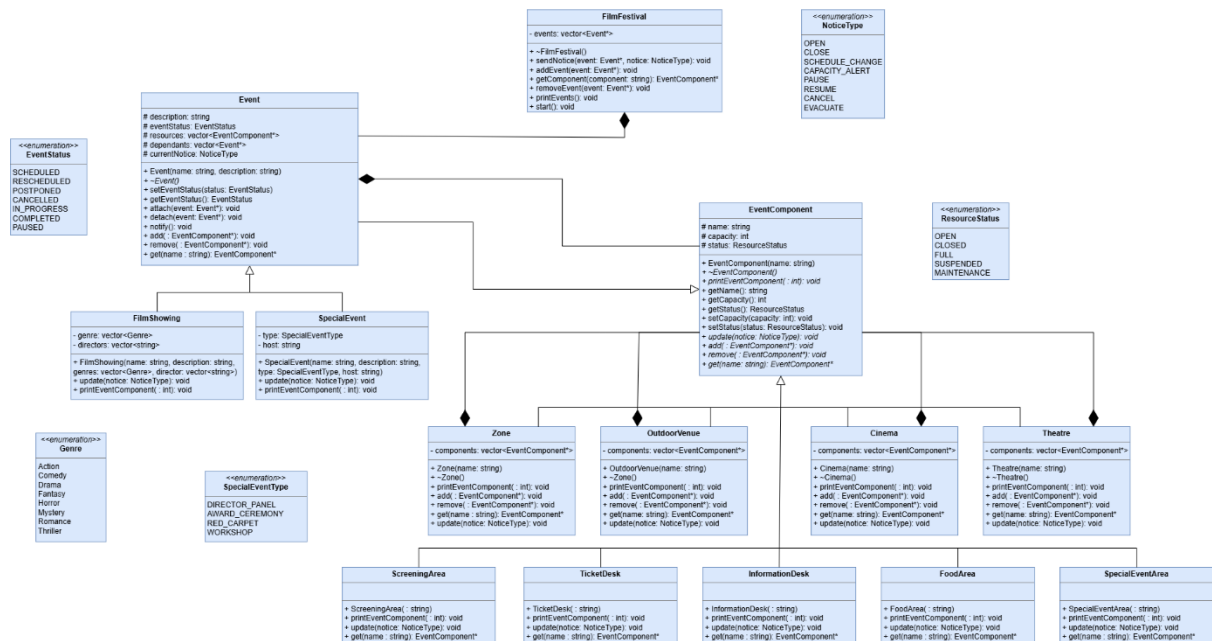
ConcreteObserver – FilmShowing, SpecialEvent

Subject – Event

ConcreteSubject - FilmShowing, SpecialEvent

(Observer is a Subject in our model)

Event is a participant in both, because an Event can depend on other Events, and if an Event gets cancelled (for example), then it has to notify the Events that depend on it (Observer and Subject).



1.4)

a) Our Composites are essentially venues, and venues can hold other venues. A common example is a cinema being inside a mall. As for leaves, a venue can contain a food area, a ticket desk, etc., but those leaves cannot have other leaves or venues. A ticket desk cannot have another ticket desk inside of it.

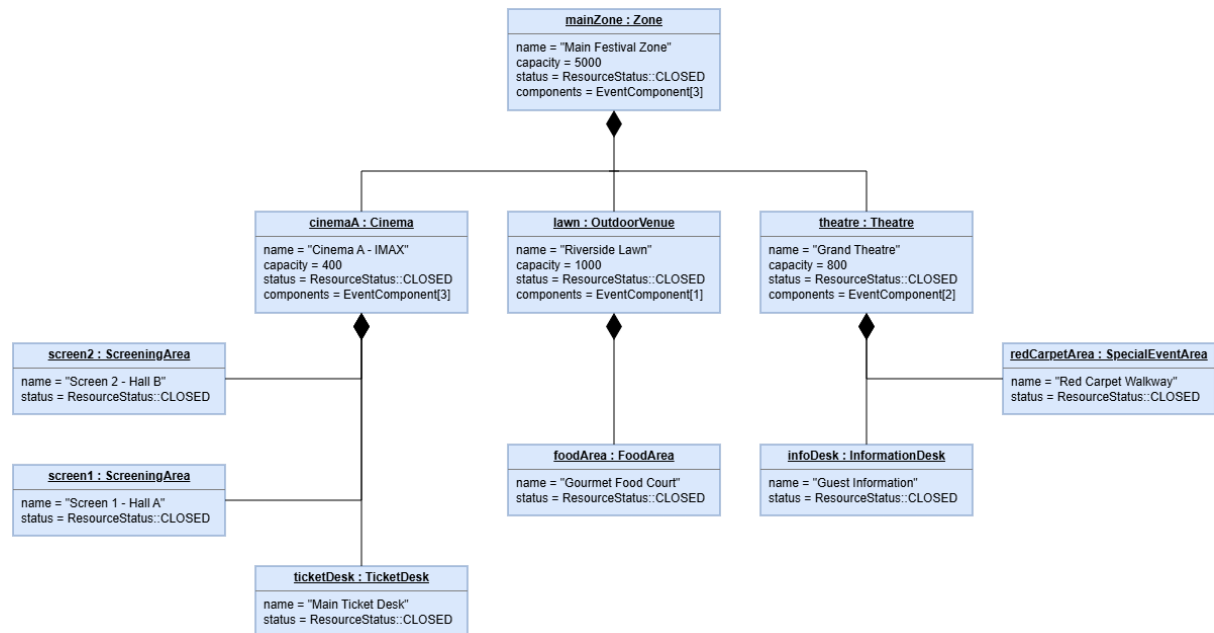
b) Since an Event is also a Composite, it can have other Events inside it. If one parent Event is cancelled/rescheduled, its children and dependents would need to be notified of this in order to decide if they should also cancel/reschedule. Hard-coded calls will not work as effectively because the client would need to keep track of which Events need to be notified after one changes, which would become needlessly complicated and easily prone to error.

c) Event components are owned by the composites, which include the Event and all types of venues.

d) The subject does not own the Observer reference, but since the Subject and Observers are part of the composite tree, Observers are owned by their parents in the Composite tree.

e) An Observer is simply a class whose objects (may) update when a Subject's object does. If an Observer object updates based on another object in the same class, then it just means that the Observer and Subject belong to the same class.

2.3)



2.4) The Composite is responsible for deleting its own child ergo ensuring that the subtrees are released exactly once. If the client were responsible for deleting the subtrees, complications could occur, resulting in a high likelihood of segfaults and similar such errata.

3.5) We chose the push approach for our Observer implementation. When the Subject changes its state, the Subject directly passes the relevant information to each registered Observer through `update(NoticeType)`. The pushed information is the `NoticeType`, which Observers then respond to. We chose push because the Observers will always need to know what `NoticeType` was passed to the Subject, so it's not necessary to query the Subject to determine what changed. This makes the notification interaction straightforward and allows components to react immediately and differently to the same notice through polymorphism.

The trade-off is that the Subject must decide what information is relevant and include it in the notification. This can make the notification interface more tightly coupled to the information being communicated than the pull approach, where Observers query the Subject themselves.

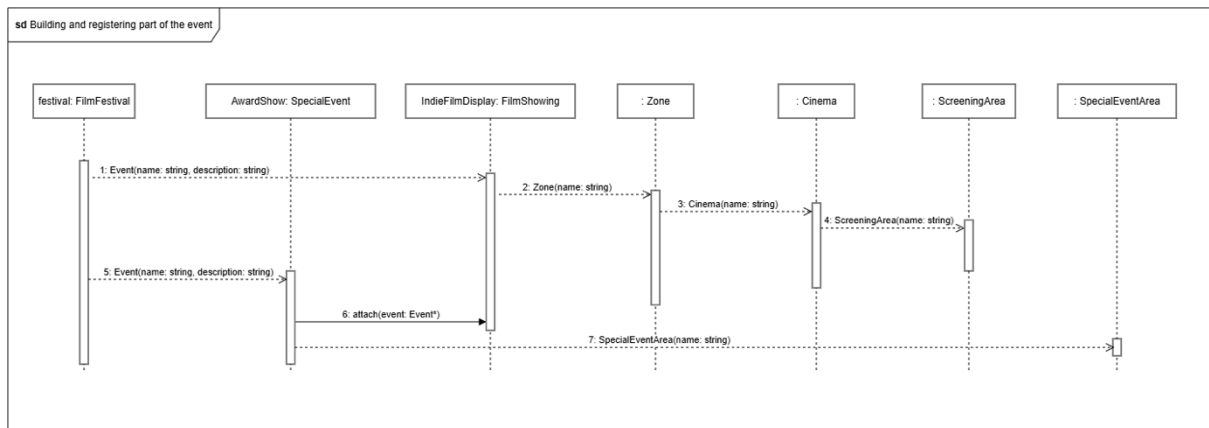
4.1)

- Assume a FilmShowing Event has a DirectorPanel as a SpecialEvent. If that FilmShowing is cancelled, the associated DirectorPanel should also be cancelled, however, every other FilmShowing should continue as normal.
- Suppose a mall is hosting a FilmFestival but is showing it at two locations at once, one in a cinema and one outdoors in the parking lot (both inside the mall's property). If a WeatherAlert occurs due to bad weather, the OutdoorVenue should be notified such that it can close and the people inside can be relocated to the Cinema; the Cinema should not close, however it should be aware of the new people coming in and send a CapacityAlert if there is not enough space.
- When a FilmFestival inside of an independent building closes/finishes, then the venues should remain open in case other movies are playing (unrelated to the festival), the TicketDesk, InformationDesk, FoodArea and ScreeningArea remain open, however the SpecialEventArea closes.
- If the FilmFestival is held inside a building used exclusively for Film Festivals, then if there is no active FilmShowing or SpecialEvent happening, the ScreeningArea, SpecialEventArea and TicketDesk should not be open. The InformationDesk and FoodArea may still be open, for if information about a previous/future festival needs to be acquired, or if someone wishes to purchase food.
- If a FilmShowing becomes Paused, then the ScreeningArea should pause and only resume when the FilmShowing resumes. The TicketDesk, InformationDesk and FoodArea remain open and active while this happens. The SpecialEventArea may need a ScheduleChange when this happens, depending on how long the FilmShowing is paused for.

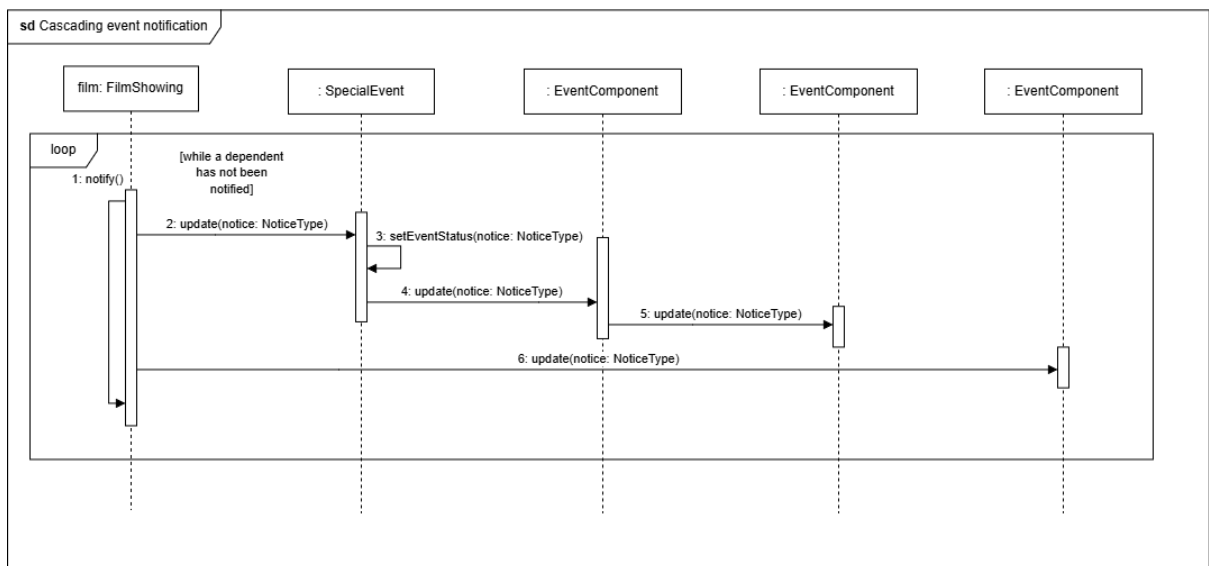
4.4)

- Event hierarchy — Introduces FilmShowing and SpecialEvent as distinct event types with their own attributes and behaviour, rather than treating every activity as a generic event component. This allows film screenings and festival activities such as Q&As or award ceremonies to behave differently.
- FilmFestival — Adds a top-level object representing the entire festival. It coordinates the overall event structure, schedule and events, providing a single entry point for managing the festival without putting all responsibilities into one class.
- Enumeration classes were added to various classes, which makes our design more compact, streamlined and efficient. This eliminates human error.

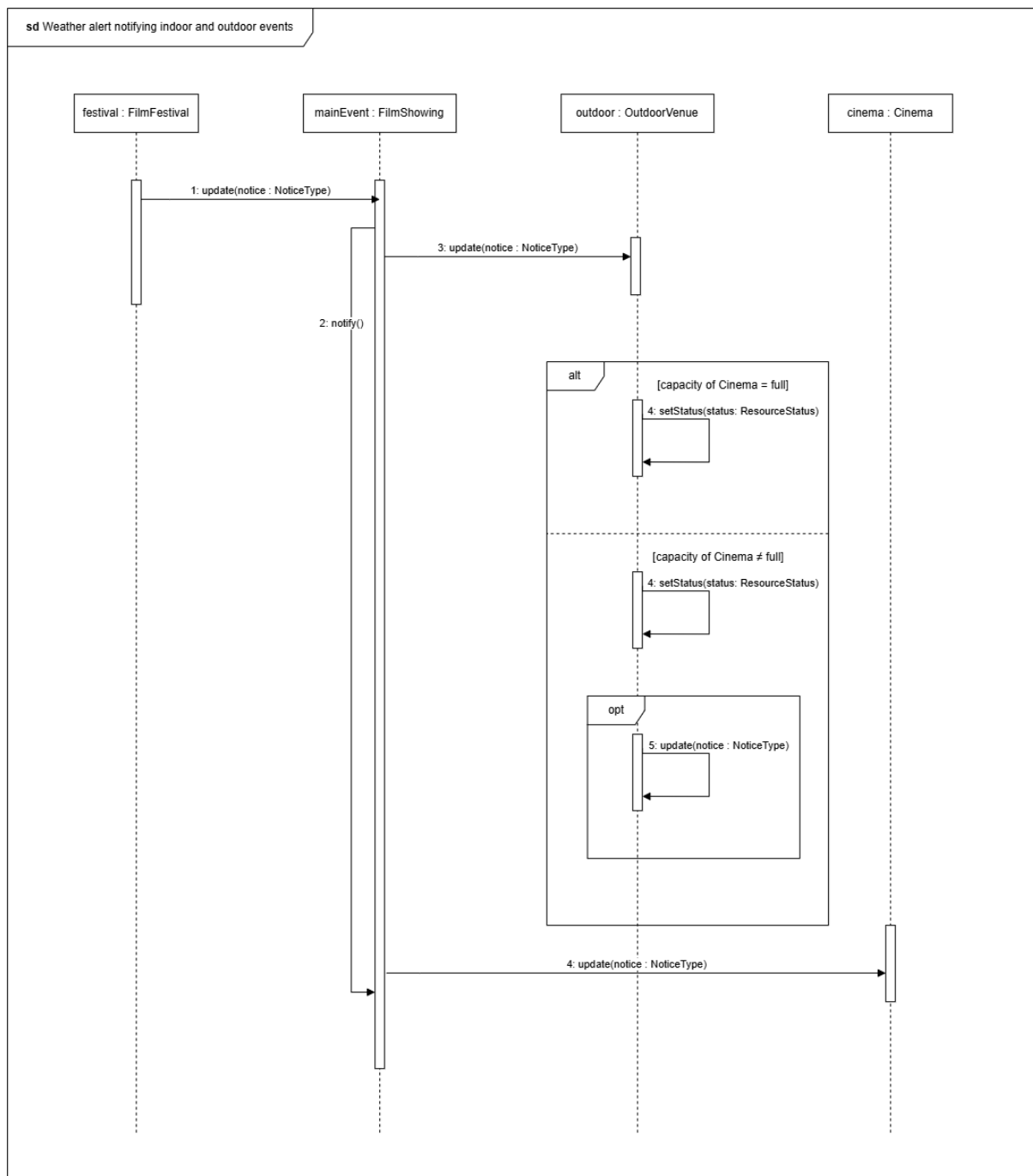
5.1)



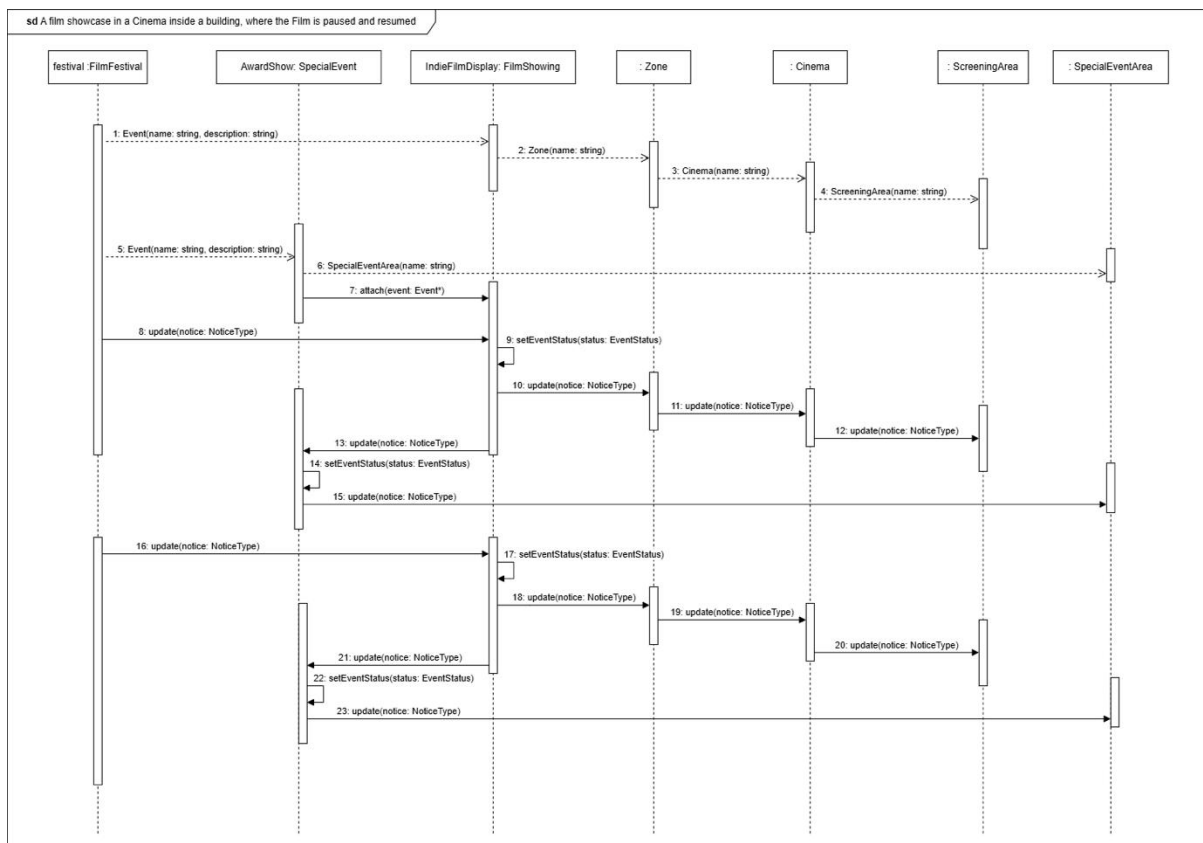
5.2)



5.3)



5.4)



Doxygen

EventFlow
COS214 Practical 3: a film-festival event coordination engine

Main Page Classes Files Search

Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Cinema	Composite-pattern Composite: a multi-screen indoor cinema building. Owns its children
Event	Abstract festival event. Doubles as both Subject and Observer: an Event can notify() its dependants (other Events observing it, e.g. a panel that depends on a screening) and is itself updated via update(NoticeType) when a notice is sent to it (e.g. by FilmFestival::sendNotice())
EventComponent	Composite-pattern Component. Represents anything that can hold or be a physical/operational resource in the festival (a whole venue, a zone, or a single desk/area)
FilmFestival	Top-level coordinator (EventControl). Owns the list of Events
FilmShowing	Concrete Event: a scheduled screening of one or more films
FoodArea	Composite-pattern Leaf: a food/vendor area
InformationDesk	Composite-pattern Leaf: a guest information/help point
OutdoorVenue	Composite-pattern Composite: an open-air venue (e.g. an outdoor screening lawn). Owns its children
ScreeningArea	Composite-pattern Leaf: a room/area where a film is screened
SpecialEvent	Concrete Event: a non-screening event such as a panel, award ceremony, red carpet walk, or workshop
SpecialEventArea	Composite-pattern Leaf: an area used for panels, red carpet, award ceremonies, and similar special events
Theatre	Composite-pattern Composite: a theatre/hall used for special events such as award ceremonies. Owns its children
TicketDesk	Composite-pattern Leaf: a ticket sales/collection point
Zone	Composite-pattern Composite: a general grouping area that can contain any mix of Leaves or further Composites

Github

https://github.com/KatNikKing/COS-214-Practical_

7.4) The work was essentially divided into coding and designing, with the designing taking two people. Simnikiwe and Junayd were in charge of the system design and diagrams, and Conrad was in charge of the coding/implementation and Doxygen. All

members, however, had a say in each aspect of the practical.

Committing helped us collaborate as a member who wants to see update some code can modify the most recent code that was committed. We also collaborated by using the “Issues” tab, which allowed us to remember what needs to be implemented and notify others of bugs/errors that may have been found.

We used a different branch for each team member, and it helped by making it easier to locate certain files that a member was working on. It also helped when changing code, as one member could experiment with the code before pushing it to main, meaning we were not at risk of overriding our already functioning code.